

Executive Product Summary

Concise product scope, priorities, architecture and first milestone

1. Product in One Sentence

A production-grade, organization-based multi-tenant SaaS API for project, task and team collaboration, built initially as an engineering-first learning project with a path toward future commercialization.

2. Current Scope

- Authentication: registration, login, JWT, refresh tokens and session management.
- Organizations: membership, invitations and organization-level RBAC.
- Projects: creation, membership and project-level authorization.
- Tasks: status, priority, assignee, due date, labels, creator and comments.
- Query capabilities: pagination, filtering, sorting and PostgreSQL-backed search.
- Infrastructure: PostgreSQL, Redis, Kafka, background processing and email notifications.
- Operations: structured logging, Prometheus, Grafana, OpenTelemetry, health/readiness and graceful shutdown.
- Quality: unit, integration, API/E2E, load and security testing.
- Delivery: Docker and CI/CD.

3. Priority Summary

Priority	Focus
P0	Foundation, authentication, organizations, RBAC, projects, tasks, comments, tenant isolation and API conventions.
P1	Query infrastructure, Redis, Kafka/background jobs, notifications and observability.
P2	E2E/load/security testing, Docker, CI/CD and production hardening.
P3	Commercial features: file storage, 2FA, OAuth/SSO, billing, audit and advanced membership.

4. Architecture Decision

Use a modular monolith first. PostgreSQL is the durable system of record. Redis handles selected distributed state and performance concerns. Kafka handles asynchronous domain events and background processing. Observability is built in rather than added after development.

5. First Milestone

The first milestone is a secure end-to-end workflow: register -> login -> create organization -> assign roles -> create project -> create task -> assign task -> comment. Advanced infrastructure is introduced around that stable vertical slice.

6. Product Principles

- Tenant isolation is a product requirement, not merely an implementation detail.
- Do not introduce infrastructure without a concrete reason.
- Prefer a coherent modular monolith over premature microservices.

- Authorization, validation, observability and testing are part of feature completion.
- Build for future scale without prematurely optimizing infrastructure.
- Keep commercial features outside the current MVP boundary.